SW개발/HW제작 설계서

프로젝트 명 : AI와 IoT를 활용한 에너지 절약 ESG 시스템 구축

2025. 10. 30

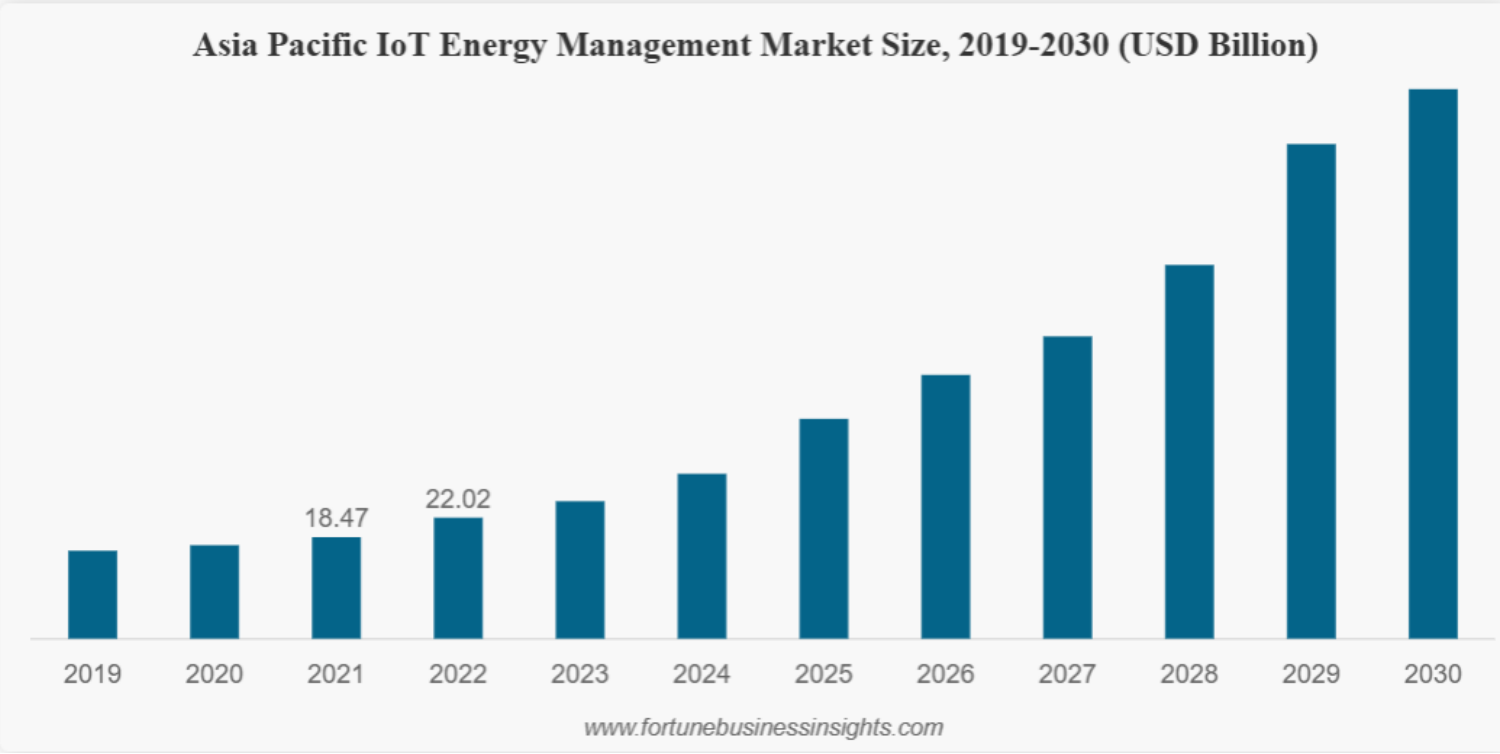
팀명: NetZeroX

수행 단계별 주요 산출물

단계	산출물	일반	응용 소프트웨어	응용 하드웨어
		·모바일 APP ·Web 등	·빅데이터 ·인공지능 ·블록체인 등	·IoT ·로봇 ·드론 등
환경 분석	시장/기술 환경 분석서	△	△	△
	설문조사 결과서	△	△	△
	인터뷰 결과서	△	△	△
요구사항 분석	요구사항 정의서	○	○	○
	유즈케이스 정의서	▲	▲	▲
아키텍처 설계	서비스 구성도(시스템 구성도)	○	○	○
	서비스 흐름도(데이터 흐름도)	△	○	△
	UI/UX 정의서	△	△	△
	하드웨어/센서 구성도	-	-	○
기능 설계	메뉴 구성도	○	○	○
	화면 설계서	○	○	△
	엔티티 관계도	○	○	△
	기능 처리도(기능 흐름도)	○	○	○
	알고리즘 명세서/설명서	△	○	○
	데이터 수집처리 정의서	-	○	-
	하드웨어 설계도	-	-	○
개발 / 구현	프로그램 목록	○	○	○
	테이블 정의서	○	○	△
	핵심 소스코드	○	○	○

※ ○ 필수, △ 선택

| 시장/기술 동향 분석



▪ IoT 에너지 관리 시장은 2022년 610억 2000만 달러 규모로 본격적인 성장세에 진입한 것으로 평가된다.

▪ 시장조사기관 Fortune Business Insights에 따르면 이 시장은 2023년 705억 8000만 달러를 시작으로 연평균 17.8% 성장하여 2030년에는 2225억 6000만 달러에 이를 것으로 추정된다.

출처 : https://www.fortunebusinessinsights.com/iot-energy-management-market-108576

| 시장/기술 동향 분석

구분	FMS (Facility Management System)	스마트 플러그 (Smart Plug)	본 프로젝트 (AI 기반 에너지 관리)
시장 규모	2021: 약 80억 USD (약 10조 원)	2024: 약 30억 USD (약 4조 원)	-
주요 수요처	대형 빌딩, 병원, 공장, 공공시설	가정, 소규모 사무실, 카페	가정, 소형 사업장, 교육기관, 연구소 등 중소규모 환경 전반
채택 장벽	높은 초기 구축비용·전문인력 필요	저렴·즉시 설치 가능	비교적 낮은 설치비용 + AI·ESG 기능 내장으로 가치 대비 ROI 우수
확장 가능성	고정 인프라 중심, 커스터마이징 시 비용 상승	Wi-Fi/BLE만 있으면 손쉬운 추가 장치 설치	모듈형 아키텍처(Docker, NCP) + 클로드 API로 센서·서비스 확장 용이

| 요구사항 정의서

구분	기능	설명
S/W	전력사용량 보기	사용한 전력 데이터의 수치를 그래프화하여 보여준다.
	하드웨어와 서버 통신	하드웨어와 서버가 통신하여 데이터를 서버에 저장한다.
	전력사용량 패턴 추천	그동안 사용한 전력사용량 패턴을 바탕으로 최적화된 전력패턴을 추천한다.
	ESG 보고서 생성	수집된 데이터를 바탕으로 LLM모델을 사용하여 ESG 보고서를 생성한다.
	AI 학습 기반 최적화	사용자 생활 패턴에 따라 Ai가 자동으로 절전 전략을 학습하고 적용한다.(적용?)
	AI를 활용한 전력기기 추론	전력사용량을 보고 학습한 데이터를 바탕으로 사용중인 전력기기를 추론한다.
	에너지 소비 패턴 시각화	일간/주간/월간 단위로 소비 패턴의 흐름을 분석하고, 이상 징후나 급증요인을 시각화한다.
	전력 이상 탐지 및 사고 예방 기능	전압 불안정, 누설 전류, 이상온도 등의 품질 관련 데이터를 수집/분석 한다.
	보안 통신 처리	MQTT 기반의 암호화된 통신으로 데이터 위조나 도청을 방지
	기상데이터 연동	외부 날씨 데이터를 연동하여 상황에 맞는 전력사용패턴을 추천한다.(???)

구분	기능	설명
H/W	전력 수집	센서를 이용해 전력 사용량을 수집한다.
	온습도 수집	센서를 이용해 온습도 정보를 수집한다.
	조도 수집	센서를 이용해 밝기를 수집한다.
	WIFI 연결	WIFI AP와 WPS를 이용하여 연결된다.
	NTP 연결 및 시간 수신	NTP 서버를 이용하여 시간 정보를 수신한다.
	MQTT 전송	각 수집된 data를 종합하여 MQTT protocol을 이용해 전송한다.
	상태 LED	LED 점등 및 깜빡임을 이용해 현재 Error code를 표시함
	전력 차단/연결	제어 정보를 수신 받고, 전력을 차단/연결한다.

| 유즈케이스 정의서

항목	설명		
개요	전력사용량 실시간 모니터링 : 가정·사무실 내 각 기기의 현재 소비 전력을 측정하여 사용자 대시보드에 실시간 갱신해 보여준다.		
관련 엑터	주엑터	사용자	
	보조 엑터	전력측정센서, ESP32모듈, 웹서버, DB	
우선순위	상	중요도	★★★★★
		난이도	중
선행조건	센서(ACS712, PZEM-004T 등)와 ESP32가 정상 동작 중이어야 한다.		
후행조건	ESP32가 Wi-Fi를 통해 서버에 연결되어 있어야 한다.		
시나리오	기본 시나리오	1. 사용자가 웹 대시보드에 접속한다. 2. 서버는 ESP32로부터 수집된 전력 데이터를 DB에서 조회한다. 3. 대시보드 UI에 실시간 그래프(또는 숫자)로 전력 사용량을 표시한다. 4. 일정 주기(예: 5초)마다 데이터 갱신을 반복한다.	
	대안 시나리오	A1: 센서 이상 응답 시 1. 서버가 ESP32 요청에 응답이 없으면 오류 메시지를 대시보드에 표시한다. 2. 사용자는 '연결 상태 확인' 버튼을 눌러 재시도할 수 있다.	
비기능 요구사항	1. 데이터 갱신 주기는 최대 5초 이내여야 한다. 2. 웹 UI 응답 시간은 200ms 이내여야 한다.		

| 유즈케이스 정의서

항목	설명		
개요	소비 패턴 자동 분석 : 수집된 전력 사용 데이터를 기반으로 장치별·시간대별 소비 패턴을 자동으로 분석하고, 에너지 절약을 위한 맞춤형 권장사항을 생성한다.		
관련 엑터	주엑터	시스템(Seq2Points·U-Net 기반 NILM 전력 시계열 분해 엔진)	
	보조 엑터	데이터베이스, 분석 서버, 사용자	
우선순위	상	중요도	★★★★★
		난이도	상
선행조건	1. 1일 이상의 전력 사용 데이터가 DB에 누적되어 있어야 한다. 2. ML 분석 모델이 학습을 완료하여 배포된 상태여야 한다.		
후행조건	분석 결과(소비 패턴 리포트 및 권장사항)가 DB에 저장되고, 사용자에게 알림 또는 대시보드로 제공된다.		
시나리오	기본 시나리오	**1. 스케줄러**가 매일(또는 설정된 주기) 정해진 시간에 "소비 패턴 분석" 작업을 트리거한다. 2. 시스템은 지난 N일(예: 7일)간의 전력 사용 데이터를 DB에서 조회한다. 3. ML 분석 엔진이 데이터를 전처리 및 특징 추출(feature extraction)한다. 4. 분석 엔진이 장치별·시간대별 소비 패턴을 클러스터링 또는 시계열 모델을 사용해 분석한다. 5. 분석 결과를 바탕으로 에너지 절약을 위한 권장사항(예: "심야 시간대 대기 전력 차단 권고", "일별 최대 사용 시간대 조정 제안")을 생성한다. 6. 권장사항과 주요 통계(평균 사용량, 최고 사용 기기 등)를 리포트 형태로 저장한다. 7. 시스템은 리포트를 사용자 대시보드에 업데이트하고, 이메일 또는 푸시 알림으로 사용자에게 전송한다.	
	대안 시나리오	**A1: 데이터 부족 시** 1. 최근 N일간 데이터 양이 모델 최소 요구치(예: 7일치)보다 부족할 경우, 시스템이 "데이터 부족" 상태를 DB에 기록한다. 2. 사용자에게 "충분한 데이터가 누적되지 않아 분석을 수행할 수 없습니다." 메시지를 알림으로 전송한다. **A2: 분석 서버 오류 시** 1. 분석 작업 중 서버 오류(메모리 부족, 네트워크 단절 등)가 발생하면, 시스템이 오류 로그를 기록한다. 2. 관리자에게 이메일(또는 SMS)로 장애 알림을 발송하고, 사용자에게 "일시적 오류로 분석을 완료하지 못했습니다." 메시지를 보여준다.	
비기능 요구사항	**1. 성능**: 하루치 데이터 분석 시간은 5분 이내여야 한다. **2. 신뢰성**: 분석 작업 실패율은 0.1% 미만이어야 한다. **3. 확장성**: 데이터 규모가 10배 증가해도 분석 시간 또는 자원 사용량 증가 폭은 선형 이하여야 한다.		

| 유즈케이스 정의서

항목	설명		
개요	이상 소비 알림 : 전력 사용량 데이터를 실시간/주기적으로 감시하여 비정상적으로 높은 소비(예: 누전, 과전류)가 감지되면 사용자에게 즉시 경고 알림을 보낸다.		
관련 엑터	주엑터	시스템(알고리즘 모듈)	
	보조 엑터	전력측정센서(ACS712), ESP32 모듈, 서버, 푸시 알림 서비스(모바일 푸시 또는 이메일)	
우선순위	상	중요도	★★★★
		난이도	중상
선행조건	1. 센서 및 ESP32가 정상 동작 중이며 주기적으로 전력 데이터를 서버에 전송 중이어야 한다. 2. 정상 소비 패턴에 대한 학습 모델(Threshold)이 배포되어 있어야 한다.		
후행조건	비정상 패턴 감지 로그가 DB에 저장되고, 사용자에게 알림이 성공적으로 전송된다.		
시나리오	기본 시나리오	1. **모니터링 서비스**가 설정된 주기(몇분으로 함???)마다 최신 전력 사용 데이터를 DB에서 조회한다. 2. 알고리즘 모듈이 조회된 데이터를 정상 소비 기준(통계적 임계치--> ewma알고리즘)과 비교 분석한다. 3. 현재 측정값이 임계치를 초과하거나 이상 패턴(급격한 상승·하락 등)이 감지되면 "이상 소비" 상태로 판단한다. 4. 시스템은 이상 소비 이벤트를 DB에 기록하고, 사용자에게 푸시 알림(API 호출) 또는 이메일을 발송한다. 5. 사용자 UI(웹/앱) 대시보드에도 경고 배너 또는 알림 아이콘이 표시된다.	
	대안 시나리오	**A1: 데이터 전송 지연 시** 1. ESP32가 서버로 데이터를 전송하지 못해 최신 데이터가 없으면, 모니터링 서비스는 "데이터 수신 지연" 로그를 기록한다. 2. 시스템은 사용자에게 "데이터 수신 지연" 알림을 별도 발송하거나, 대시보드에 상태 메시지를 표시한다.-->흠 **A2: 알림 전송 실패 시** 1. 푸시/이메일 서비스 API 호출 오류 발생 시, 시스템이 재시도 로직(예: 최대 3회)을 수행한다. 2. 모든 재시도 실패 시 관리자 이메일로 장애 알림을 발송하고, 사용자 UI에 "알림 전송 실패" 상태를 표시한다.	
비기능 요구사항	1. **경고 지연 시간**: 이상 소비 감지 후 알림 발송까지 30초 이내여야 한다. 2. **신뢰성**: 이상 이벤트 누락률은 0.1% 미만이어야 한다. 3. **확장성**: 동시에 10개 이상의 센서를 모니터링할 수 있어야 한다 4. **모니터링**: 이상탐지 기록을 대시보드로 제공해야 한다.		

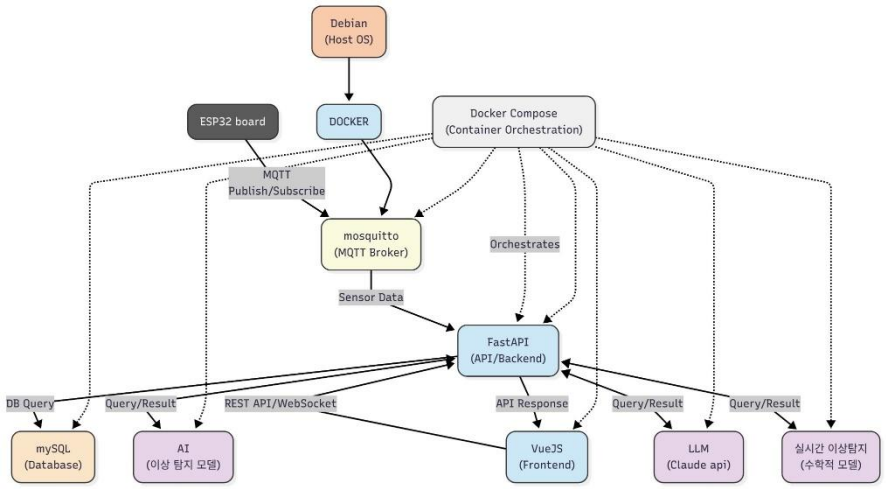
| 유즈케이스 정의서

항목	설명--여기부터다시		
개요	ESG 보고서 자동 생성 : 월별·분기별·연단위로 수집된 에너지 사용량, 탄소 배출량, 비용 절감 효과 등을 종합하여 기업·기관용 ESG 평가 보고서를 자동으로 생성하고 배포한다.		
관련 엑터	주엑터	관리자	
	보조 엑터	시스템(리포트 엔진), 데이터베이스, 문서 생성 서비스(PDF/Excel), 이메일/파일 서버	
우선순위	중상	중요도	★★★★
		난이도	중
선행조건	1. 지정된 기간(예: 지난 한 달) 동안의 에너지 사용량 및 탄소 배출 데이터가 DB에 완전히 누적되어 있어야 한다. 2. ESG 보고서 템플릿(디자인·항목 정의)이 시스템에 등록되어 있어야 한다. 3. 관리자 계정이 시스템에 로그인되어 있어야 한다.		
후행조건	생성된 보고서 파일이 스토리지에 저장되고, 관리자에게 이메일 알림 또는 대시보드 다운로드 링크가 제공된다.		
시나리오	기본 시나리오	1. 관리자가 웹 대시보드에서 "ESG 보고서 생성" 메뉴를 선택한다. 2. 시스템은 보고서 기간(예: YYYY-MM) 및 템플릿 옵션(포맷, 포함 항목 등)을 입력받는다. 3. 시스템은 DB로부터 지정 기간의 에너지 사용량, 탄소 배출량, 비용 절감 통계 데이터를 조회한다. 4. 리포트 엔진이 템플릿에 데이터를 삽입하여 PDF 및 Excel 형식의 보고서를 생성한다. 5. 생성된 보고서 파일이 파일 서버(또는 클라우드 스토리지)에 업로드된다. 6. 시스템은 관리자에게 "보고서 생성 완료" 이메일을 발송하고, 대시보드에 다운로드 링크를 표시한다.	
	대안 시나리오	**A1: 데이터 조회 오류 시** 1. DB 연결 오류 또는 데이터 누락 발생 시, 시스템이 오류 로그를 기록하고 관리자 UI에 "데이터 조회 실패" 메시지를 표시한다. 2. 관리자는 오류 원인을 확인하고 재시도 버튼을 눌러 보고서 생성을 다시 시도할 수 있다. **A2: 템플릿 미등록 시** 1. 지정한 템플릿이 시스템에 존재하지 않으면, 시스템이 "템플릿 없음" 오류를 관리자에게 알린다. 2. 관리자는 템플릿 관리 메뉴에서 새 템플릿을 업로드한 뒤 보고서를 생성한다. **A3: 문서 생성 실패 시** 1. PDF 또는 Excel 변환 과정에서 라이브러리 오류가 발생하면, 시스템이 재시도 로직을 수행한다(최대 2회). 2. 재시도 후에도 실패하면, 관리자에게 "보고서 생성 실패" 이메일을 발송하고 로그를 남긴다.	
비기능 요구사항	1. **성능**: 보고서 생성(데이터 조회 → 파일 생성 → 업로드) 전체 과정이 2분 이내에 완료되어야 한다. 2. **확장성**: 연간 누적 데이터가 10배 증가해도 보고서 생성 시간이 선형 이하로 증가해야 한다. 3. **신뢰성**: 생성 실패율은 0.5% 미만이어야 하며, 실패 시 자동 재시도 및 관리자 통보가 이루어져야 한다. 4. **유지보수성**: 새로운 보고서 항목(지표)이 추가될 경우 템플릿과 리포트 엔진 설정만으로 즉시 반영할 수 있어야 한다.		

| 유즈케이스 정의서

항목	설명		
개요	외부 데이터(날씨) 연동 : 기상청 또는 민간 기상 API로부터 실시간 기온, 습도, 일조량 등의 기상 정보를 주기적으로 수집하여 시스템 분석 모델 및 대시보드에 반영한다.		
관련 엑터	주엑터	시스템(백엔드 통합 모듈)	
	보조 엑터	외부 날씨 API 제공자, 데이터베이스, 분석 서버, 대시보드 UI	
우선순위	중	중요도	★★★
		난이도	중
선행조건	1. 외부 날씨 API 사용을 위한 인증 키(API Key)가 발급되어 있어야 한다. 2. 백엔드 서버가 인터넷에 연결되어 있어야 한다. 3. 스케줄러가 설정된 주기에 따라 동작 준비가 되어 있어야 한다.		
후행조건	수집된 날씨 데이터가 DB에 저장되고, 분석 엔진 및 대시보드가 최신 기상 정보를 반영하도록 업데이트된다.		
시나리오	기본 시나리오	**1. 스케줄러**가 설정된 주기(예: 1시간마다)마다 날씨 연동 작업을 트리거한다. 2. 백엔드 통합 모듈이 외부 날씨 API에 REST 호출을 통해 현재 기상 정보를 요청한다. 3. API 제공자가 JSON/XML 형식으로 기온, 습도, 일조량, 강수 확률 등의 데이터를 응답한다. 4. 모듈은 응답 데이터를 파싱하여 내부 데이터 구조로 변환한 뒤, DB에 저장한다. 5. 분석 서버가 저장된 기상 데이터를 불러와 에너지 소비 예측 모델 및 절약 권장사항 생성 로직에 반영한다. 6. 대시보드 UI는 최신 날씨 정보를 조회하여 화면의 날씨 위젯 및 그래프에 표시한다.	
	대안 시나리오	**A1: API 호출 실패 시** 1. 네트워크 오류 또는 인증 실패로 응답이 없으면, 모듈이 재시도 로직(최대 3회)으로 호출을 재시도한다. 2. 재시도 후에도 실패하면, 오류 로그를 기록하고 관리자에게 이메일/SMS 알림을 발송한다. 3. 기존에 저장된 가장 최신 날씨 데이터를 대시보드에 표시하되 "실시간 연동 오류" 배지를 함께 표시한다. **A2: 데이터 포맷 변경 시** 1. API 제공자가 응답 스키마를 변경하여 파싱 오류가 발생하면, 모듈이 스키마 불일치 로그를 남긴다. 2. 개발자는 API 스펙에 맞춰 파싱 로직을 업데이트하고 배포하여 연동을 복구한다. 3. 시스템은 복구 후 백필(backfill) 작업을 통해 누락된 데이터를 재수집한다.	
비기능 요구사항	**1. 주기성**: 기본 연동 주기는 1시간마다, 필요 시 10분 단위로 조정 가능해야 한다. **2. 신뢰성**: 연동 실패율은 1% 미만이어야 하며, 실패 시 자동 재시도 및 관리자 알림이 수행되어야 한다. **3. 확장성**: 향후 대기오염, 대기질 지수(AQI) 등의 추가 외부 API도 동일 프레임워크로 쉽게 추가할 수 있어야 한다. **4. 모니터링**: 연동 작업의 상태(성공/실패, 응답 시간 등)를 별도 모니터링 대시보드에서 조회할 수 있어야 한다.		

| 서비스 구성도 - 서비스 시나리오



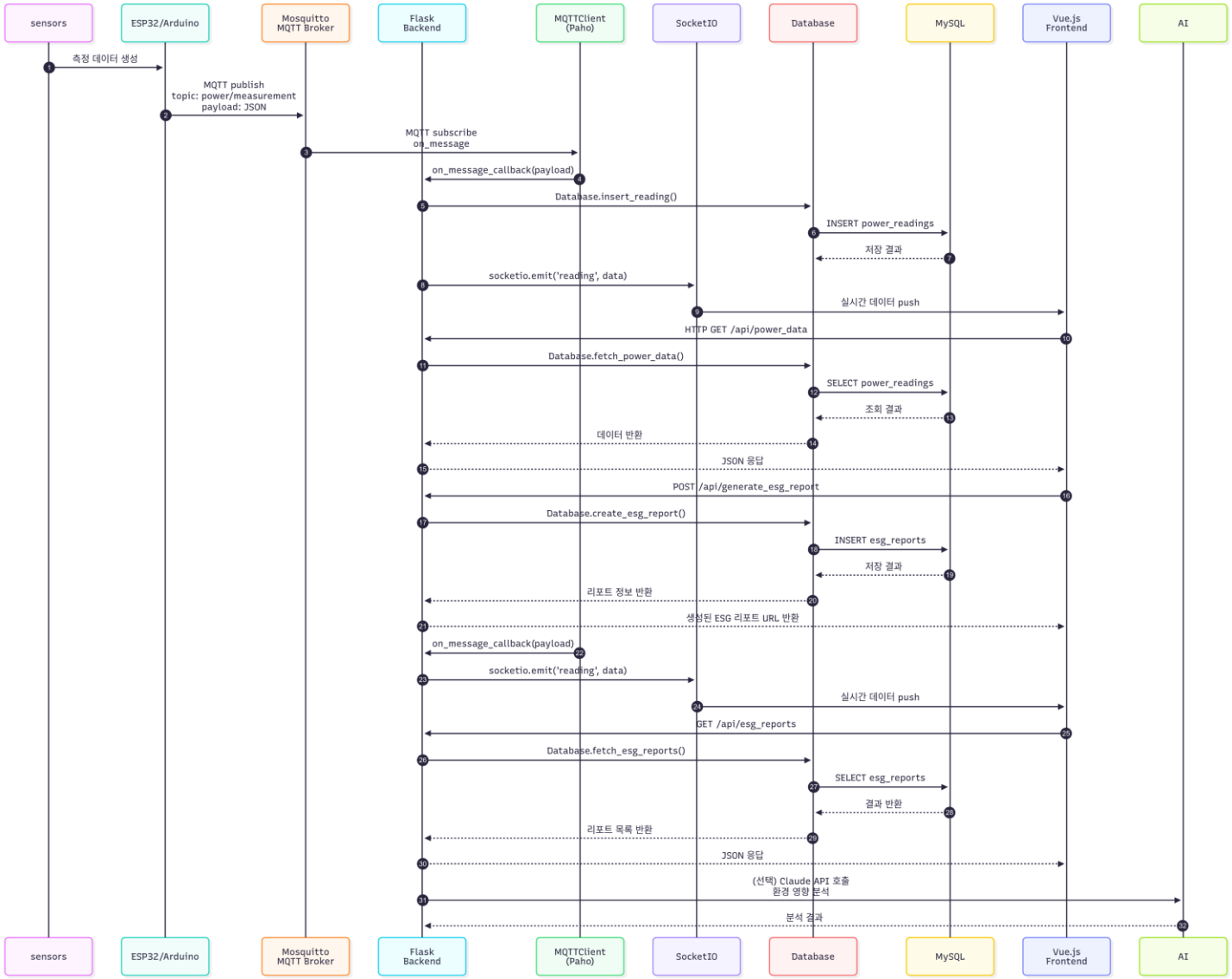
Back-end

- **센서 데이터 수집**
  - ESP32 → MQTT 브로커 → 백엔드 수신
  - 수신된 전력, 온습도, 등을 실시간으로 파싱하여 저장
- **데이터 저장 및 관리**
  - MySQL에 전력 사용량, 탄소 배출량, 센서 메타정보 기록
  - 신규 기기 등록·제거, 사용자 계정·권한 테이블 관리
- **AI 분석 및 예측**
  - Scikit-learn / TensorFlow 모델로 과거 사용 패턴 학습
  - 향후 24시간 전력 수요 예측·비용 최적화 스케줄 생성
- **ESG 보고서 자동 생성**
  - Claude.API를 호출해 분석 요약문 작성
  - PDF 리포트 렌더링 후 다운로드 제공

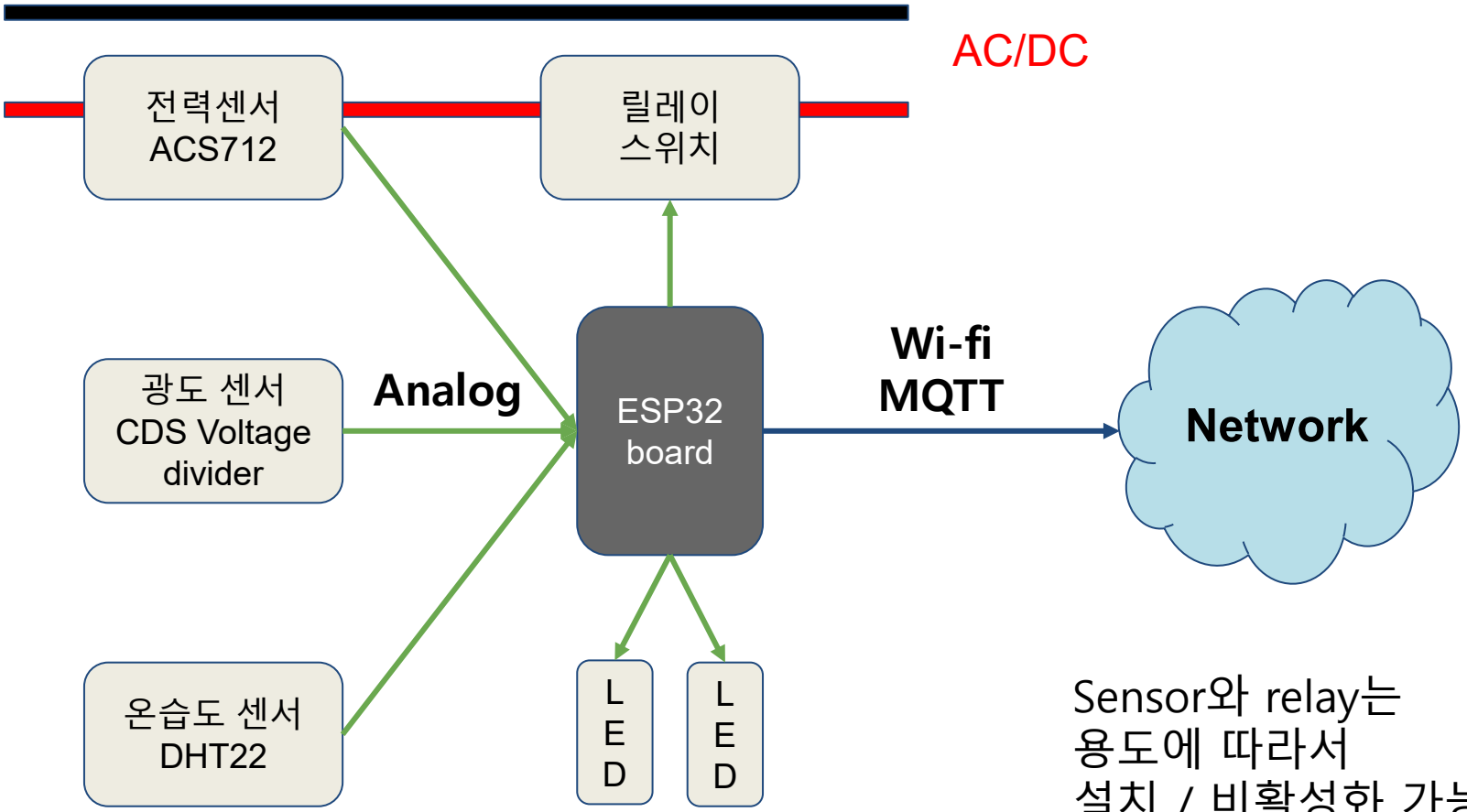
Front-end

- **실시간 대시보드 렌더링**
  - WebSocket/MQTT을 통해 실시간 사용량 게이지·차트 갱신
  - 연결 상태, 알림 배너(이상 탐지·요금 급등 예보) 표시
- **소비 패턴 조회 및 추천 탭**
  - 일/주/월 소비 추이 라인차트, AI 추천 스케줄 리스트 제공
  - 추천 스케줄 "적용" 버튼 클릭 시 Back end API 호출
- **ESG 리포트 뷰어**
  - 생성된 PDF 목록 조회, 인 앱 미리보기
  - 다운로드·이메일 전송 요청
- **설정 및 사용자 관리**
  - 디바이스 등록/삭제, 알림 임계치(경고 레벨) 설정
  - 사용자 권한 부여(관리자·일반) 및 프로필 편집

| 서비스 흐름도


sensors
ESP32/Arduino
Mosquitto MQTT Broker
Flask Backend
MQTTClient (Paho)
SocketIO
Database
MySQL
Vue.js Frontend
AI
측정 데이터 생성
MQTT publish
topic: power/measurement
payload: JSON
MQTT subscribe
on_message
on_message_callback(payload)
Database.insert_reading()
INSERT power_readings
저장 결과
socketio.emit('reading', data)
실시간 데이터 push
HTTP GET /api/power_data
Database.fetch_power_data()
SELECT power_readings
조회 결과
데이터 반환
JSON 응답
POST /api/generate_esg_report
Database.create_esg_report()
INSERT esg_reports
저장 결과
리포트 정보 반환
생성된 ESG 리포트 URL 반환
on_message_callback(payload)
socketio.emit('reading', data)
실시간 데이터 push
GET /api/esg_reports
Database.fetch_esg_reports()
SELECT esg_reports
결과 반환
리포트 목록 반환
JSON 응답
(선택) Claude API 호출
환경 영향 분석
분석 결과

| 하드웨어 / 센서 구상도



상태 표시등

Sensor와 relay는
용도에 따라서
설치 / 비활성화 가능

| 메뉴 구성도

```
스플래시 화면── 대시보드 ├── 실시간 모니터링
          |          |   ├── 현재 전력 사용량
          |          |   ├── 온·습도 현황
          |          |   └── 이상 탐지 알림
          |          |
          |          ├──기기 관리
          |          |   ├── 기기 목록
          |          |   |   ├── ESP32-거실
          |          |   |   └── ESP32-사무실
          |          |   └── 신규 기기 등록
          |          |
          |          └── 소비 추이 요약──── 에너지 분석──────────── 보고서
          |              ├── 일간          ├── 소비 추이            ├── ESG 보고서
          |              ├── 주간          |   ├── 일간 그래프      |   ├── 월별 보고서 생성
          |              └── 월간          |   ├── 주간 그래프      |   ├── 분기별 보고서 생성
          |                                |   └── 월간 그래프      |   └── 보고서 목록 & 다운로드
          |                                └── 패턴 추천            └── 탄소 비교
          |                                ├── AI 추천 스케줄 목록     ├── 전월 대비 차트
          |                                └── 추천 적용/해제          └── 감축 실적
          |
          └──설정──────────────────────────도움말 & 지원
               ├── 관리자 PIN 변경              ├── 사용 가이드
               ├── 데이터 보관 기간 설정        ├── 문의하기
               ├── 알림 임계치 설정             └── 버전 정보
               └── 공장 초기화
```

| 엔티티 관계도 - ERD

power_readings			
int	id	PK	AUTO_INCREMENT
datetime	timestamp		NOT NULL
int	device_code		NOT NULL
float	temperature		Celsius
float	humidity		Percentage
int	brightness		Numeric value
float	electric		mA
timestamp	created_at		DEFAULT CURRENT_TIMESTAMP

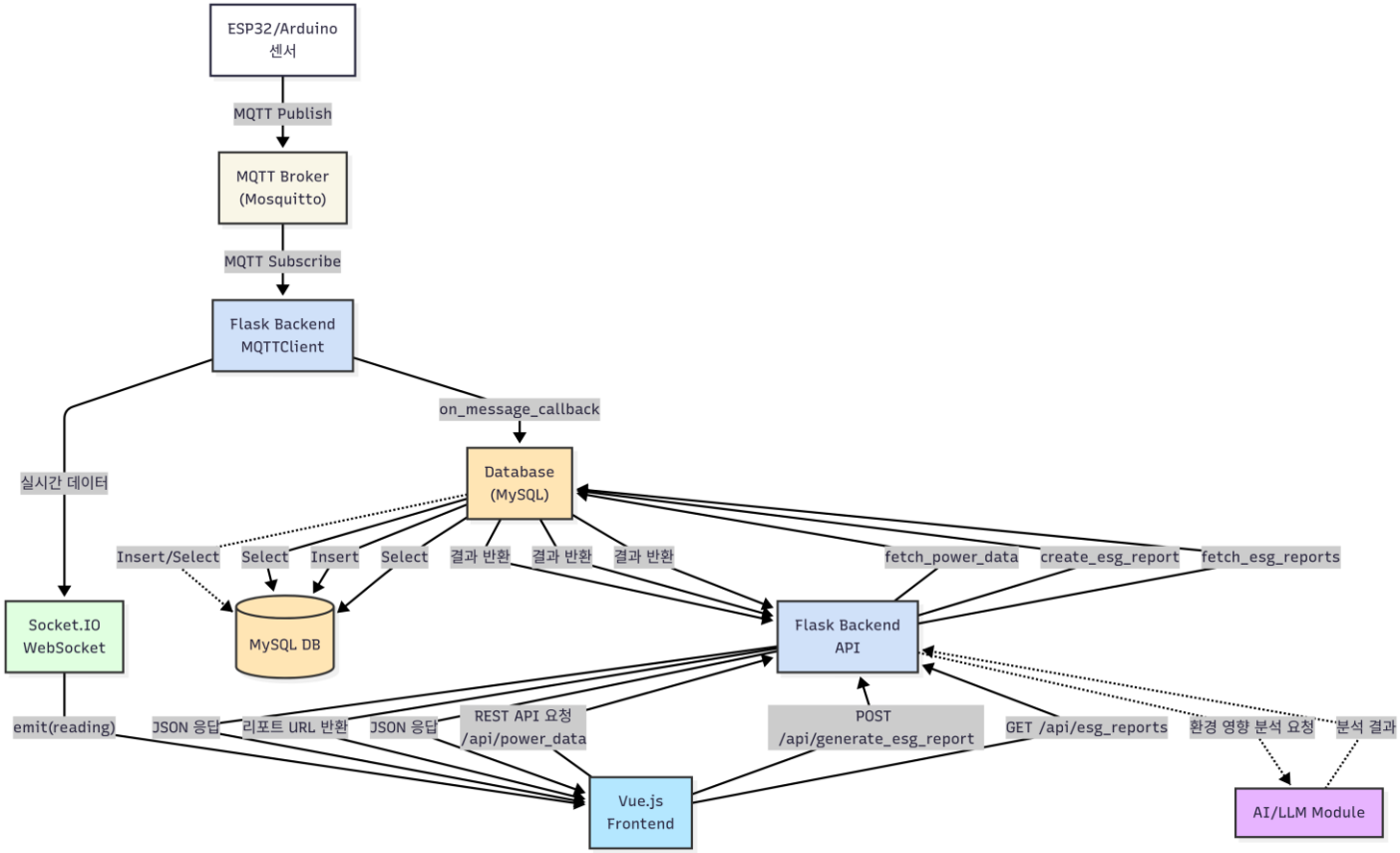
esg_reports			
int	id	PK	AUTO_INCREMENT
varchar	file_path		NOT NULL (255)
datetime	created_at		NOT NULL

| 기능 처리도(기능 흐름도)

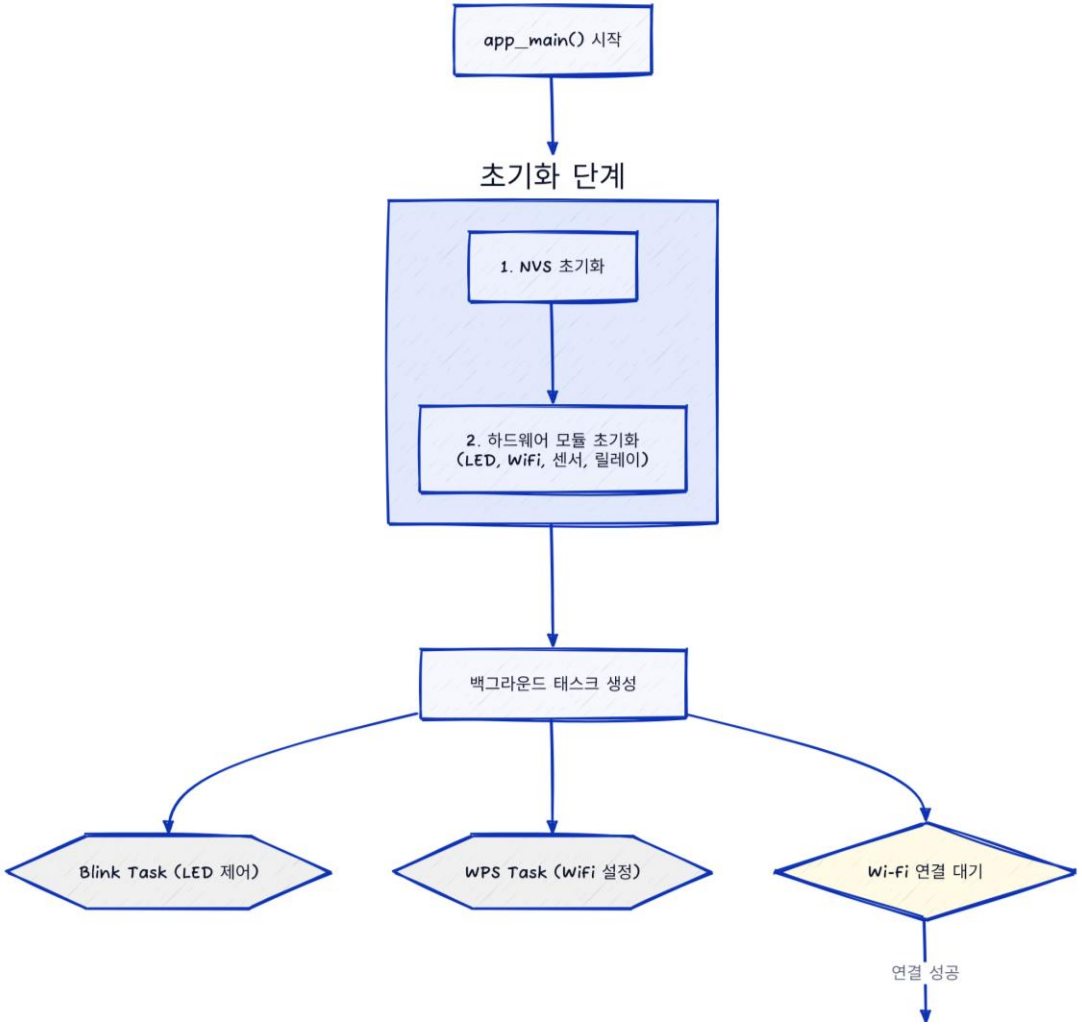
실무 산출물 형식

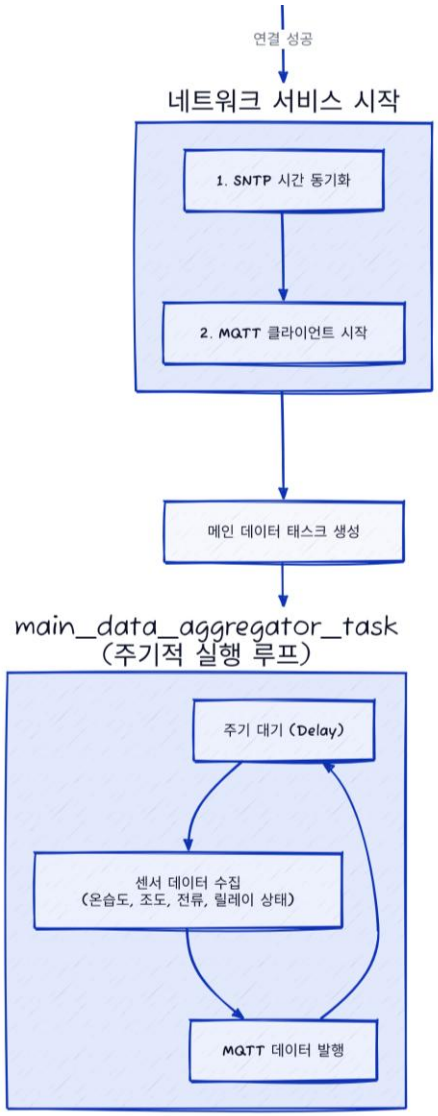
프로그램 ID	EMS-2025-001	프로그램 명	AI 기반 에너지 관리 시스템	작성일	2025.07.15.	Page	1/1
개요	ESP32 기반 IoT 센서를 통해 실시간 전력 사용 데이터를 MQTT로 수집하고, Flask 백엔드 API에서 머신러닝(TensorFlow/Scikit-learn) 모델을 활용해 사용 패턴을 분석·예측합니다. 결과는 MySQL에 저장되며, Vue.js 프론트엔드 대시보드를 통해 실시간 모니터링, AI 추천 스케줄, ESG 보고서 자동 생성, 이상 탐지 알림 등의 기능을 제공합니다. NCP(네이버 클라우드 플랫폼)에 배포된 Docker 컨테이너 환경에서 안정적으로 운영됩니다.						

기능 흐름도



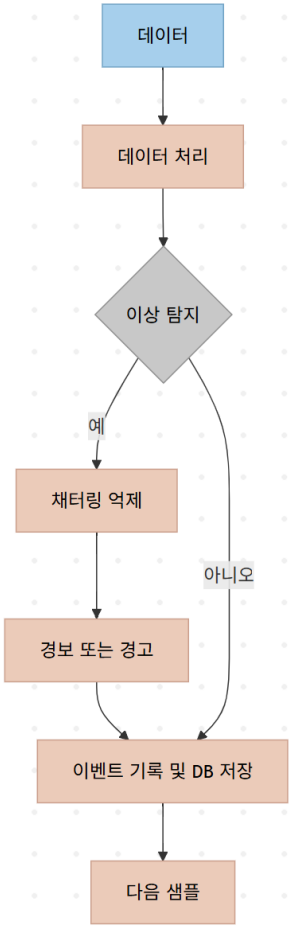
| 기능 처리도(기능 흐름도)


app_main() 시작
초기화 단계
1. NVS 초기화
2. 하드웨어 모듈 초기화
(LED, WiFi, 센서, 릴레이)
백그라운드 태스크 생성
Blink Task (LED 제어)
WPS Task (WiFi 설정)
Wi-Fi 연결 대기
연결 성공


연결 성공
네트워크 서비스 시작
1. SNTP 시간 동기화
2. MQTT 클라이언트 시작
메인 데이터 태스크 생성
main_data_aggregator_task
(주기적 실행 루프)
주기 대기 (Delay)
센서 데이터 수집
(온습도, 조도, 전류, 릴레이 상태)
MQTT 데이터 발행

| 알고리즘 명세서

1. 이상탐지



○ 알고리즘 시나리오

① 데이터 수집·정렬-센서 스트림을 시간순으로 정리하고 결측/이상값을 간단히 처리한다.

② 기준선 업데이트(EWMA)
실시간으로 기준선을 갱신하면서 패턴 이상 징후를 모니터링하고, 지속되면 이벤트로 승격한다.

③ 안전 우선 체크(전류)
현재 전류가 위험구간이면 즉시 경보, 경계구간이 일정 시간 이어지면 경고한다.

④ 급변 감지(스파이크)
전류가 갑자기 크게 변하면 즉시 스파이크 경보한다.

⑤ 환경 조건 검사
실내·실외 온도 차이를 보고 여름/겨울 규칙으로 이상 여부 판단한다.

⑥ 중복·채터링 억제
짧은 쿨다운과 해제 조건을 둬서 같은 알람의 과도한 반복을 방지한다.

⑦이벤트 정리·출력
감지된 이벤트를 우선순위(스파이크/과전류 > 패턴 > 온도)로 정렬해 알림/로그/DB에 기록하고, 다음 샘플로 반복한다.

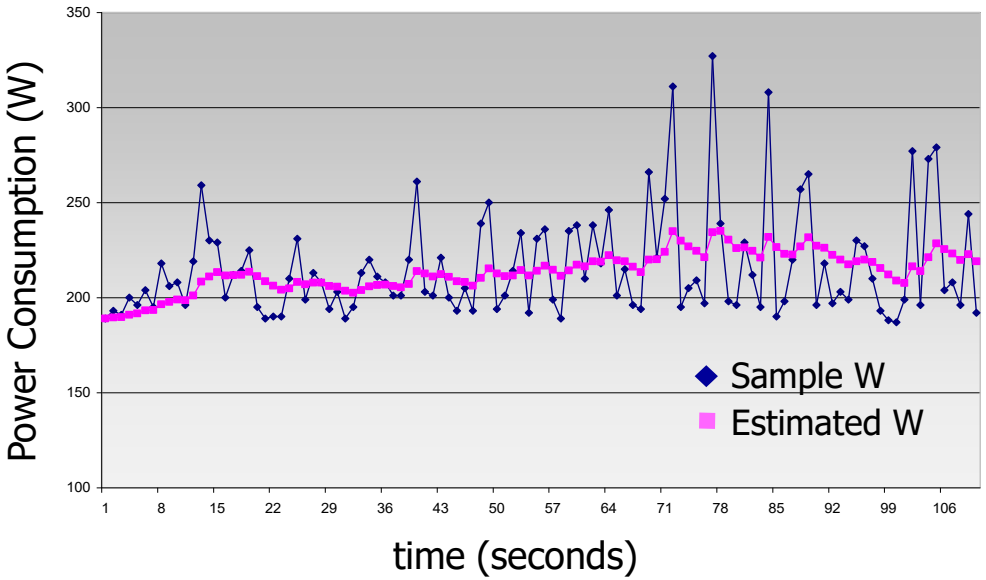
1. 이상탐지

○ **지수적 이동평균 (EWMA:Expenentialy Weighted Moving Average) 알고리즘**
간단한 수식을 이용해서 과거의 값을 이용해 다음 예측 값을 만들어낸다. 간단한 수식은 적은 연산부하로 실시간으로 연산하기에 적합하다. 과거의 값은 지수적으로 감소하므로 비교적 최근의 값들이 크게 반영되어 다음 값을 예측한다.

단기간의 에너지 사용량을 효과적으로 예측해내고, 예측 값과 새로 들어온 값을 비교하여 이상을 탐지해낼 수 있다.

- **DevW**: EWMA of **SampleW** deviation from **EstimatedW**:

```
DevW = (1-β)*DevW + β*|SampleW -EstimatedW|
```

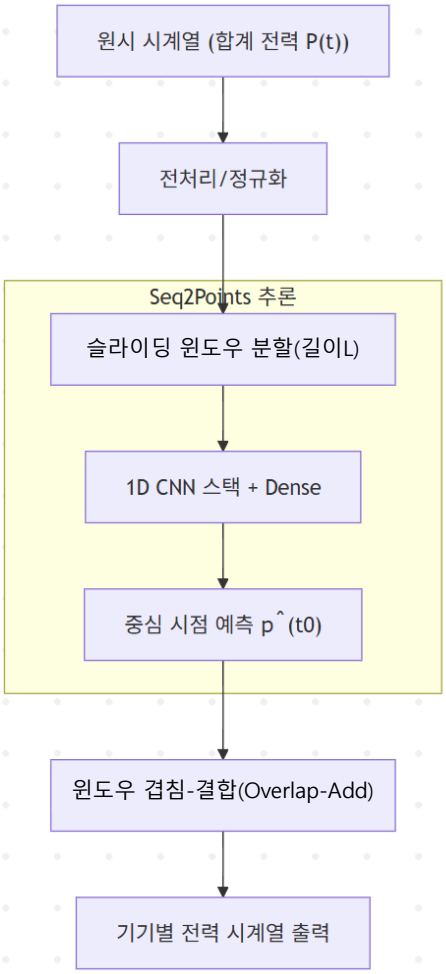


- Abnormal Consumption
  : **EstimatedW** plus "safety margin"
  - large variation in **EstimatedW**
    : want a larger safety margin

```
Abnormal Consumption
 = EstimatedW + 4*DevW
```

"safety margin"

2. 전력분석모델



○ 알고리즘 시나리오

① 데이터 폴더와 결과 폴더를 정리하고, 필요하면 GPU 사용 가능 여부를 확인한다.

② CSV 파일을 압축 형식(Parquet)으로 바꾸고 시간순으로 정렬·결측 처리한다. 학습용 평균과 표준편차를 저장해 Z-점수 정규화를 적용한다.

③ 길이 1198의 슬라이딩 창을 만들고, 각 창의 **중앙 시점** 값을 정답으로 삼는다. 메모리를 아끼기 위해 창 데이터를 캐시 형태로 보관한다.

④ 1D CNN 5층 + 완전연결 2층 구조로 중앙값을 예측하도록 학습한다. 손실은 MSE(참고로 MAE도 확인)로 두고, Adam 최적화·기울기 클리핑·조기 종료를 사용해 과적합을 막는다. 성능이 가장 좋은 가중치를 저장하고, 필요하면 여러 GPU로 속도를 높인다.

⑤ 실시간이면 길이 1198 포인트가 모일 때마다 예측한다. 파일 단위면 데이터를 읽어 예측 후 정규화를 되돌린다. 창 단위 예측은 겹쳐 이어붙여(Overlap-Add) 연속 시계열로 복원한다.

⑥ MAE로 성능을 확인하고, 기기별 전력 추정 결과에서 이벤트·사용 패턴을 추출해 대시보드, 알림, 리포트에 활용한다.

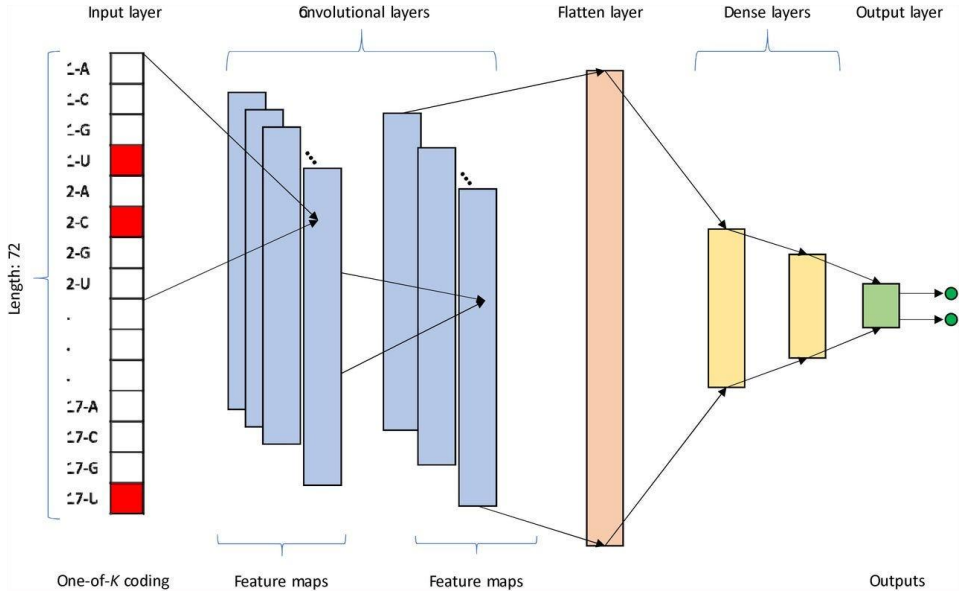
2. 소비전력기기분석

○ **seq2point 모델**

- seq2Point 모델은 전체 전력 사용량 데이터에서 특정 가전기기의 전력 소비량을 분리하는 에너지 분해(NILM)를 위한 딥러닝 모델이다.

이 모델은 1D Convolutional 레이어로 구성된 다층 CNN(Convolutional Neural Network)을 기반으로 시계열 데이터의 패턴 및 특징을 추출한다. 24h의 전체 전력 시퀀스를 입력으로 받아, 해당 시퀀스의 중앙에 위치한 단일 시점의 가전기기 전력량을 예측한다.

결과값은 시점과 전력 사용량 예측으로 나오며, 이를 종합하여 하루 전력 사용량의 전체 통계를 확인할 수 있다.

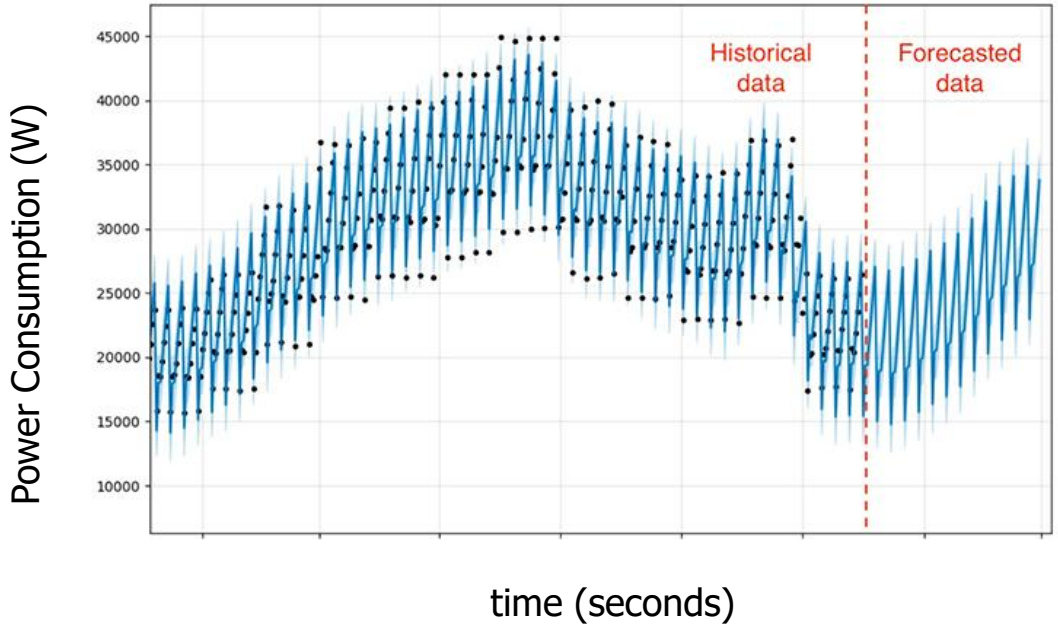


2. 소비전력기기분석

○ **시계열 예측(Time Series Forecasting) 알고리즘**

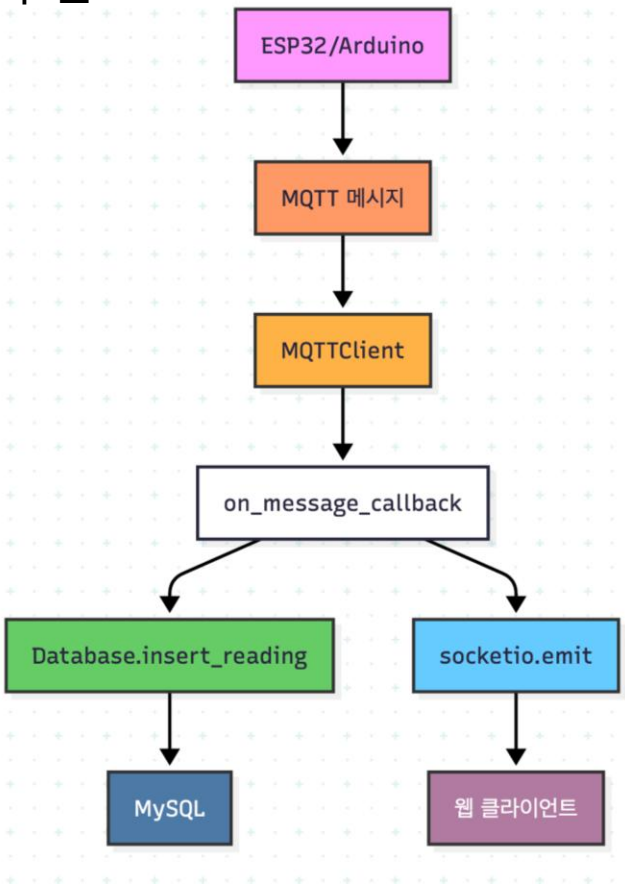
Time Series Forecasting은 현재까지 존재하는 데이터를 바탕으로 미래를 예측하는 알고리즘이다. Graph fitting을 통해서 현재의 값에 최대한 유사한 graph를 추정하여 data의 미래를 예측한다.

현재까지의 에너지 사용량을 이용하여, 사용량의 주간 패턴, 계절적 요인, 최근 경향 등을 종합적으로 분석해 월말까지의 예상 소비량을 제공한다. 이를 통해 사용자는 예측된 사용량을 사전에 확인하고, 효율적인 전력 관리 계획을 세울 수 있다.

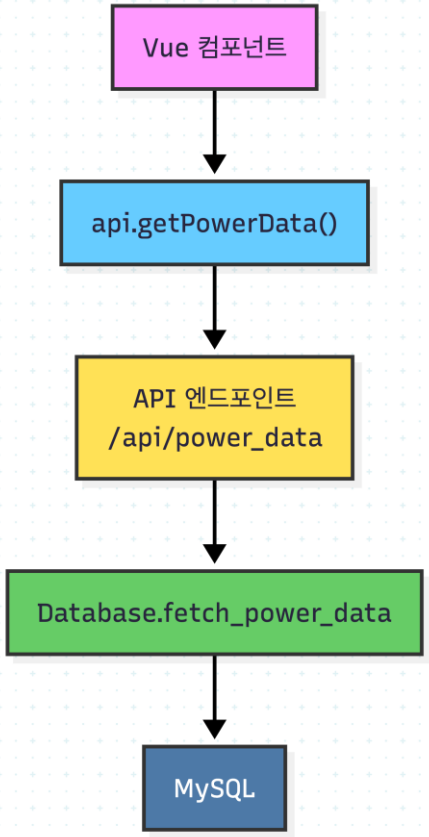


| 데이터 수집 / 처리 정의

1. 데이터 수집

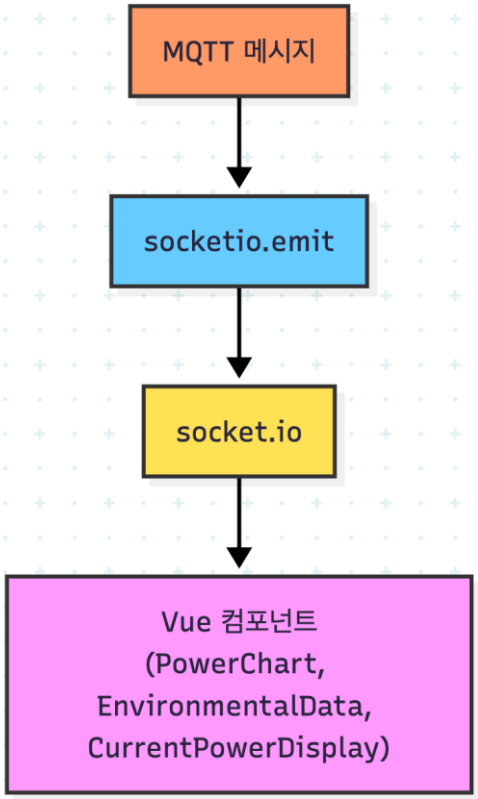


2. 데이터 조회

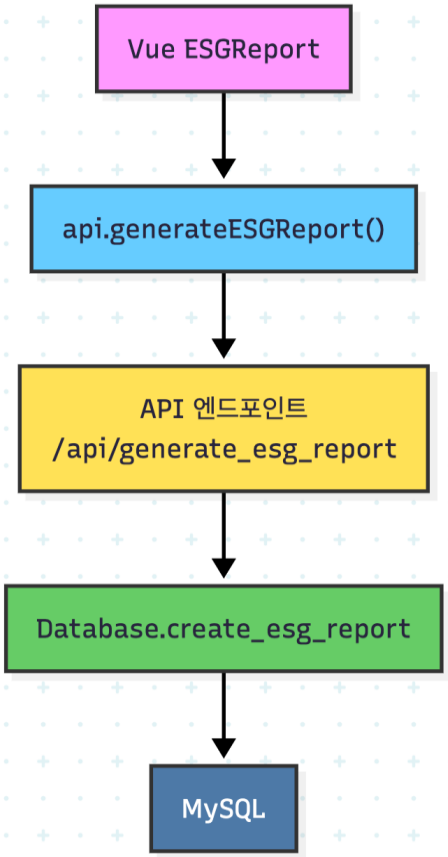


| 데이터 수집 / 처리 정의

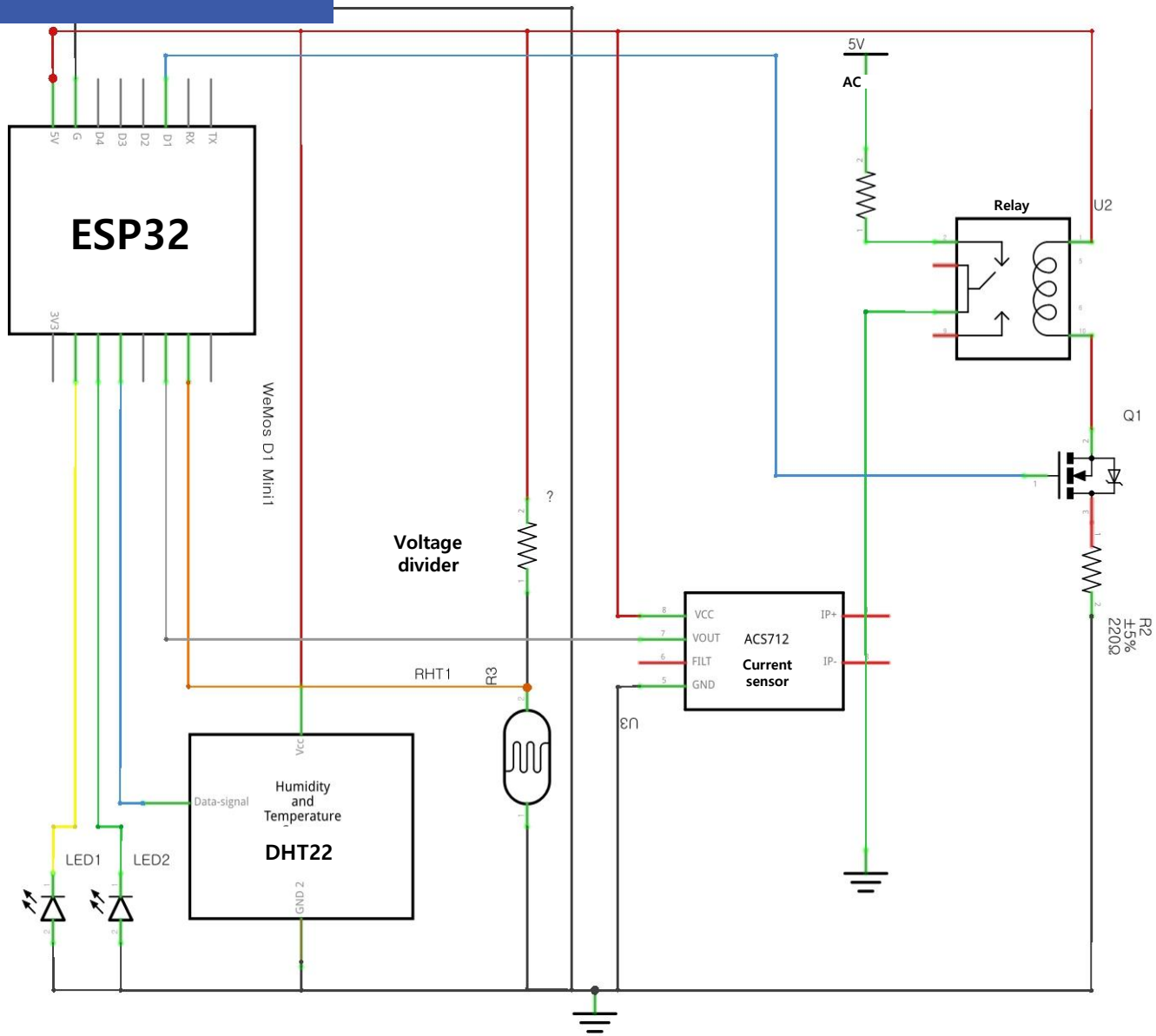
3. 데이터 수집



4. 데이터 조회



| 하드웨어 설계도


ESP32
5V
G
D4
D3
D2
D1
RX
TX
3V3
WeMos D1 Mini1
Voltage divider
?
RHT1
R3
VCC
Data-signal
Humidity and Temperature
DHT22
GND 2
LED1
LED2
VCC
VOUT
FILT
GND
ACS712
Current sensor
IP+
IP-
U3
5V
AC
Relay
U2
Q1
R2
±5%
220Ω

| 프로그램 - 목록-1

기능 분류	기능번호	기능 명
PWR	PWR-01	전력 감지 시스템
	PWR-01-01	전력 데이터 수집
	PWR-01-02	이상 징후(과전류-누전) 감지
CTRL	CTRL-02	원격 제어
	CTRL-02-01	장치 ON/OFF 제어
	CTRL-02-02	제어 스케줄관리
ANA	ANA-03	전력 사용 패턴 분석
	ANA-03-01	머신러닝 기반 패턴 학습
	ANA-03-02	최적화 방안 제안

| 프로그램 - 목록-2

기능 분류	기능번호	기능 명
UI	UI-04	사용자 인터페이스
	UI-04-01	실시간 대시보드
	UI-04-02	제어 패널(원격제어)
EXT	EXT-05	외부 데이터 연동
	EXT-05-01	날씨 정보(시온-일조량) 수집
	EXT-05-02	소비 예측 반영
RPT	RPT-06	ESG 보고서 자동 생성
	RPT-06-01	데이터 가공
	RPT-06-02	보고서 출력

| 핵심 소스코드(Flask)

Main Application

```
# Flask 앱 초기화 및 CORS 설정
app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="*")

# 데이터베이스 및 MQTT 클라이언트 초기화
db = Database()
mqtt_client = MQTTClient(on_message_callback=on_message_callback)

# MQTT 메시지 처리 콜백 함수
def on_message_callback(payload):
# JSON 파싱 및 데이터베이스 저장
# 실시간 웹소켓 클라이언트에 데이터 전송
socketio.emit('reading', {
'device_code': device_code,
'timestamp': timestamp,
'temperature': temp,
'humidity': humidity,
'brightness': brightness,
'electric': electric
})
```

```
# API 라우트 등록 및 서버 시작
api_blueprint = setup_routes(db)
app.register_blueprint(api_blueprint)
```

MQTT Client class

```
class MQTTClient:
"""MQTT 브로커와 통신하는 클라이언트 클래스
- 메시지 브로커(Mosquitto)에 연결하고 지정된 토픽을 구독
- 메시지 수신 시 콜백 함수를 호출하여 데이터 처리
- 별도 스레드에서 네트워크 루프 실행으로 비동기 처리
"""
def __init__(self, on_message_callback, topics=['power/measurement']):
# 브로커 연결 설정 및 콜백 함수 등록
def on_connect(self, client, userdata, flags, rc):
# 연결 성공 시 지정된 토픽 구독
def on_message(self, client, userdata, msg):
# 메시지 수신 시 콜백 함수에 전달
def start(self):
# 브로커 연결 및 별도 스레드에서 네트워크 루프 시작
def stop(self):
# 연결 종료 및 정리
```

| 핵심 소스코드(Vue.js)

API Service

```
// Axios 인스턴스 생성 및 API 호출 함수 정의
const apiClient = axios.create({
baseURL: '/api',
timeout: 10000,
headers: { 'Content-Type': 'application/json' }
});

// 주요 API 함수:
// getESGReports() - ESG 보고서 목록 조회
// generateESGReport() - 새 ESG 보고서 생성
// getPowerData(params) - 전력 데이터 조회 (limit, timeRange 등 파라미터 지원)
```

Web Socket

```
// Socket.IO 클라이언트 설정
// 실시간 데이터 수신을 위한 웹소켓 연결
const socket = io('/', {
path: '/socket.io',
transports: ['websocket', 'polling'],
});
```

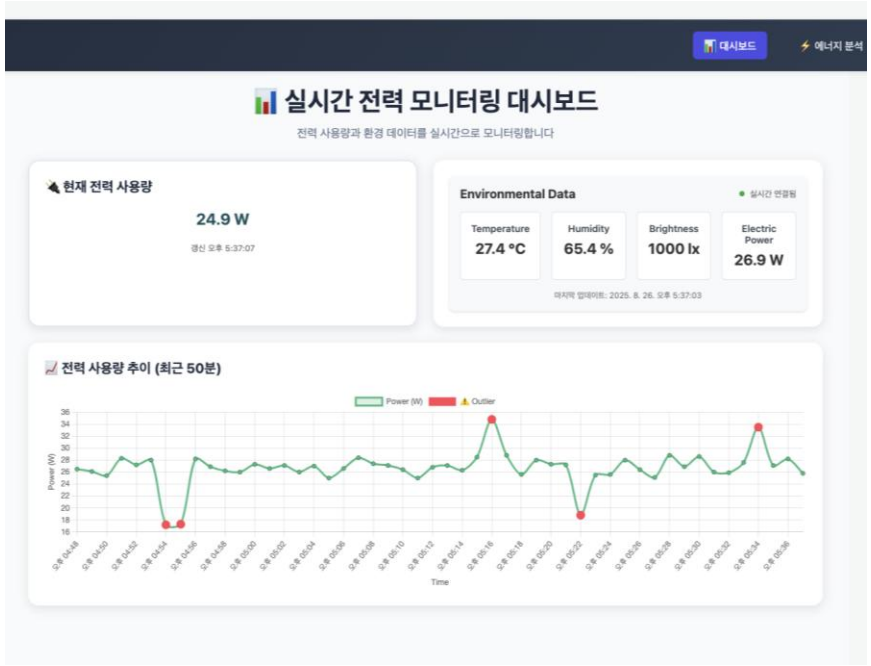
센서 모듈 Main - data aggregator task

```
static void main_data_aggregator_task(void *pvParameter) {
    // Give sensors some time to get their first reading
    vTaskDelay(pdMS_TO_TICKS(SENSOR_PERIOD_MS));
    const TickType_t interval = pdMS_TO_TICKS(PUBLISH_PERIOD_MS);
    TickType_t last_wake_time = xTaskGetTickCount();
    while (1) {
        sensor_data_t current_data = {0};
#if WATT_METER_ATTACHED
        currentFourierAnalysis(&current_data.ampere);
#else
        current_data.ampere = -999; // Placeholder
#endif
... (skip) ...
        // Publish data via MQTT
        mqtt_publish_data(&current_data);
        vTaskDelayUntil(&last_wake_time, interval);
    }
}
```

| 참조-개발 환경 및 설명

구분		항목	적용내역
S/W 개발환경	AI 기반 에너지 관리 시스템 개발	Python 3.10 / Flask	서버 및 API 처리, 데이터 전처리/후처리 구현
		Scikit-learn	전력 사용량 예측 및 이상 감지 등을 위한 머신러닝 모델 학습 및 추론에 사용
		Docker	배포 및 통합 테스트를 위한 컨테이너 기반 환경
		MySQL(5.1.73)	전력 데이터 및 사용자 패턴 저장용 데이터베이스
		Claude API	ESG 보고서 자동 생성 및 요약 응답 처리
		TensorFlow	전력 사용 패턴 분석 및 AI 예측 모델 학습
		MQTT	IoT 센서(ESP32)와 서버 간의 경량 메시징을 통한 실시간 데이터 통신
H/W 구성장비	디바이스	Free RTOS	ESP32에서 멀티태스킹 기반 센서 데이터 수집 및 통신 처리
		ESP32	전류 센서 및 센서 데이터 수집 제어용 MCU
		ACS712	전류 센서 (가정용 전력 측정)
		DHT22	온습도 센서 (환경 정보 보조 수집)
	통신	Wi-Fi	IoT 장치와 서버 간 실시간 데이터 전송
		MQTT	센서 데이터의 효율적인 실시간 전송 프로토콜
	서버	네이버 클라우드 플랫폼 (NCP)/ Local Linux Server	데이터 저장, 모델 추론, ESG 리포트 생성 서버

| 참조-S/W 기능 실사 사진



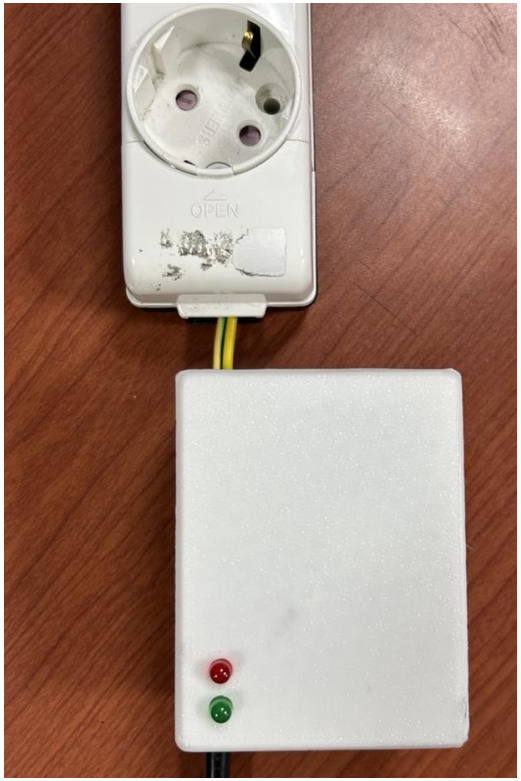
Front End 화면 test

ESG 환경보고서 조회

| 참조-H/W 기능 실사 사진

```
26 15:28:26", "temp": -999.00, "humidity": -999.00, "brightness": -999, "electric
": 0.0,"onoff": "OFF"}
I (45900) watt_meter: RMS Current at 60Hz (bin 120): 0.00 W
I (45901) watt_meter: FFT for 1024 complex points took 117644 cycles
I (45908) MQTT_HANDLER: Sent publish successful, msg_id=26293
I (45909) MQTT_HANDLER: Payload: {"deviceCode": "1234666", "timestamp": "2025-08-
26 15:28:28", "temp": -999.00, "humidity": -999.00, "brightness": -999, "electric
": 0.0,"onoff": "OFF"}
I (47014) MQTT_HANDLER: MQTT_EVENT_DATA
TOPIC=device/command
DATA={"deviceCode": "1234666", "command": "ON"}
I (47015) Relay: Relay on GPIO 17 turned ON
I (47980) watt_meter: RMS Current at 60Hz (bin 123): 9.36 W
I (47981) watt_meter: FFT for 1024 complex points took 117644 cycles
I (47988) MQTT_HANDLER: Sent publish successful, msg_id=38987
I (47989) MQTT_HANDLER: Payload: {"deviceCode": "1234666", "timestamp": "2025-08-
26 15:28:30", "temp": -999.00, "humidity": -999.00, "brightness": -999, "electric
": 9.4,"onoff": "ON"}
I (50060) watt_meter: RMS Current at 60Hz (bin 123): 19.71 W
I (50061) watt_meter: FFT for 1024 complex points took 117644 cycles
I (50070) MQTT_HANDLER: Sent publish successful, msg_id=30726
I (50071) MQTT_HANDLER: Payload: {"deviceCode": "1234666", "timestamp": "2025-08-
26 15:28:32", "temp": -999.00, "humidity": -999.00, "brightness": -999, "electric
": 19.7,"onoff": "ON"}
```

Monitoring 화면



전류 – 전원 제어 HW

| 참조-H/W 기능 실사 사진

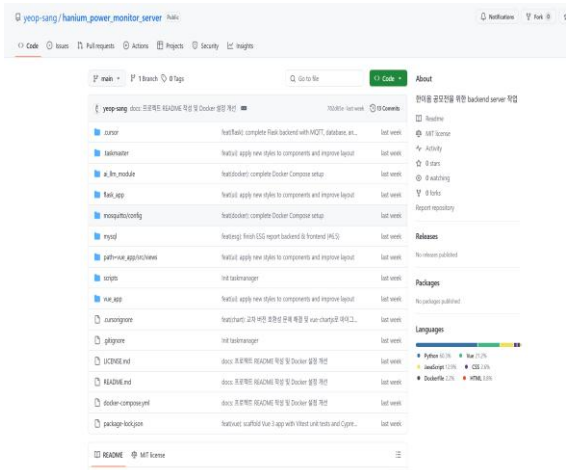
```
I (5103) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us
to 307200 us
I (5161) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (6832) esp_netif_handlers: sta ip: 192.168.0.13, mask: 255.255.255.0, gw: 1
92.168.0.1
I (6833) WIFI_HANDLER: ✅ Got IP: 192.168.0.13
I (6834) APP_MAIN: Wi-Fi Connected!
I (6837) SNTP: Initializing SNTP
I (6841) SNTP: Waiting for system time to be set... (1/15)
I (8845) SNTP: Waiting for system time to be set... (2/15)
I (10660) SNTP: Time synchronized
I (10845) SNTP: The current date/time is: Tue Aug 26 17:06:19 2025
I (10854) APP_MAIN: Initialization complete. Main loop is running.
I (10855) main_task: Returned from app_main()
I (11017) MQTT_HANDLER: ✅ MQTT Connected
I (12863) MQTT_HANDLER: Sent publish successful, msg_id=48982
I (12864) MQTT_HANDLER: Payload: {"deviceCode": "1234666", "timestamp": "2025
-08-26 17:06:21", "temp": 0.00, "humidity": 0.00, "brightness": 627, "electri
c": -999.0,"onoff": "OFF"}
I (14863) MQTT_HANDLER: Sent publish successful, msg_id=4558
I (14864) MQTT_HANDLER: Payload: {"deviceCode": "1234666", "timestamp": "2025
-08-26 17:06:23", "temp": 0.00, "humidity": 0.00, "brightness": 618, "electri
c": -999.0,"onoff": "OFF"}
```

Monitoring 화면

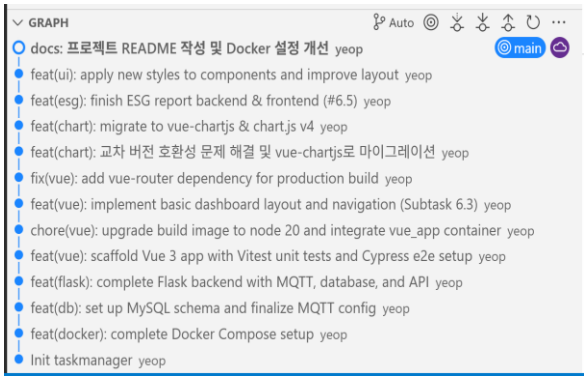
환경 수집 센서 HW

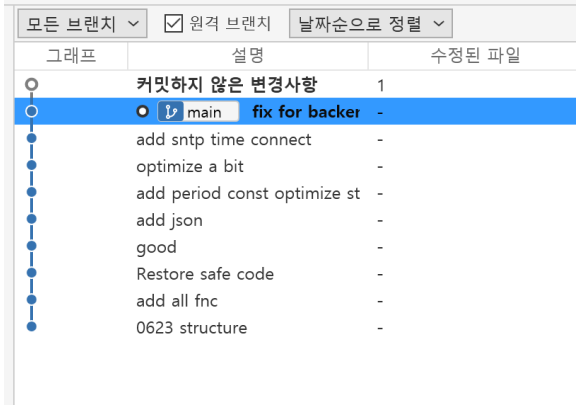
| 참조-프로젝트 관리

프로젝트 관리 - github

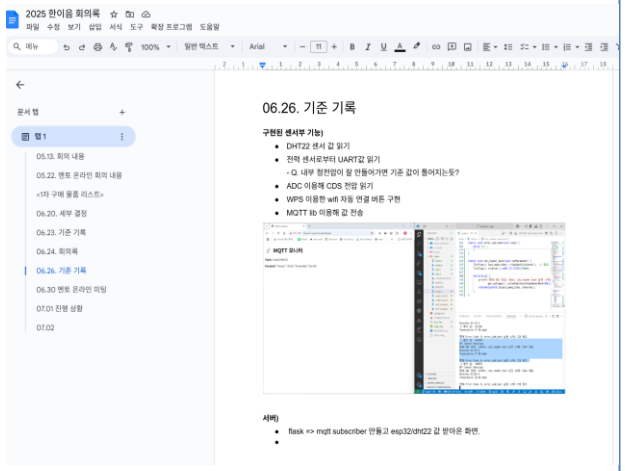


형상관리 - git





이슈관리 – google doc



Thank you